

Syslab 之 Julia 作用域

平文婷

苏州同元软控信息技术有限公司

2024-05-23



TONGYUAN
Software and Control

课程须知

本课程课程目标：

本课程面向科学计算新用户，讲解在软件使用中，Julia 语法中的作用域问题；深入了解作用域，帮助用户更好使用 Julia。

本课程基于：Syslab 2024a云化版构建

本课程课程学习前，需要学习以下前置课程：

- Syslab 基本功能
- Julia 语法

@苏州同元软控信息技术有限公司版权所有，侵权必究

目录

1. 作用域概念和意义

2. Julia 的变量作用域

3. 避免作用域引起的问题

01

作用域概念和意义

©苏州同元软控信息技术有限公司版权所有，未经许可，不得复印、传播或以其他方式使用

@苏州同元软控信息技术有限公司
版权所有，侵权必究

1.作用域概念和意义

- 变量的 **作用域** 是某个变量可见的范围，变量作用域使得同名的不同变量能够区分开来。
- 相同的变量名是否指向同一个东西的规则被称为作用域规则。
- 给变量划分作用域有助于**解决变量命名冲突**。

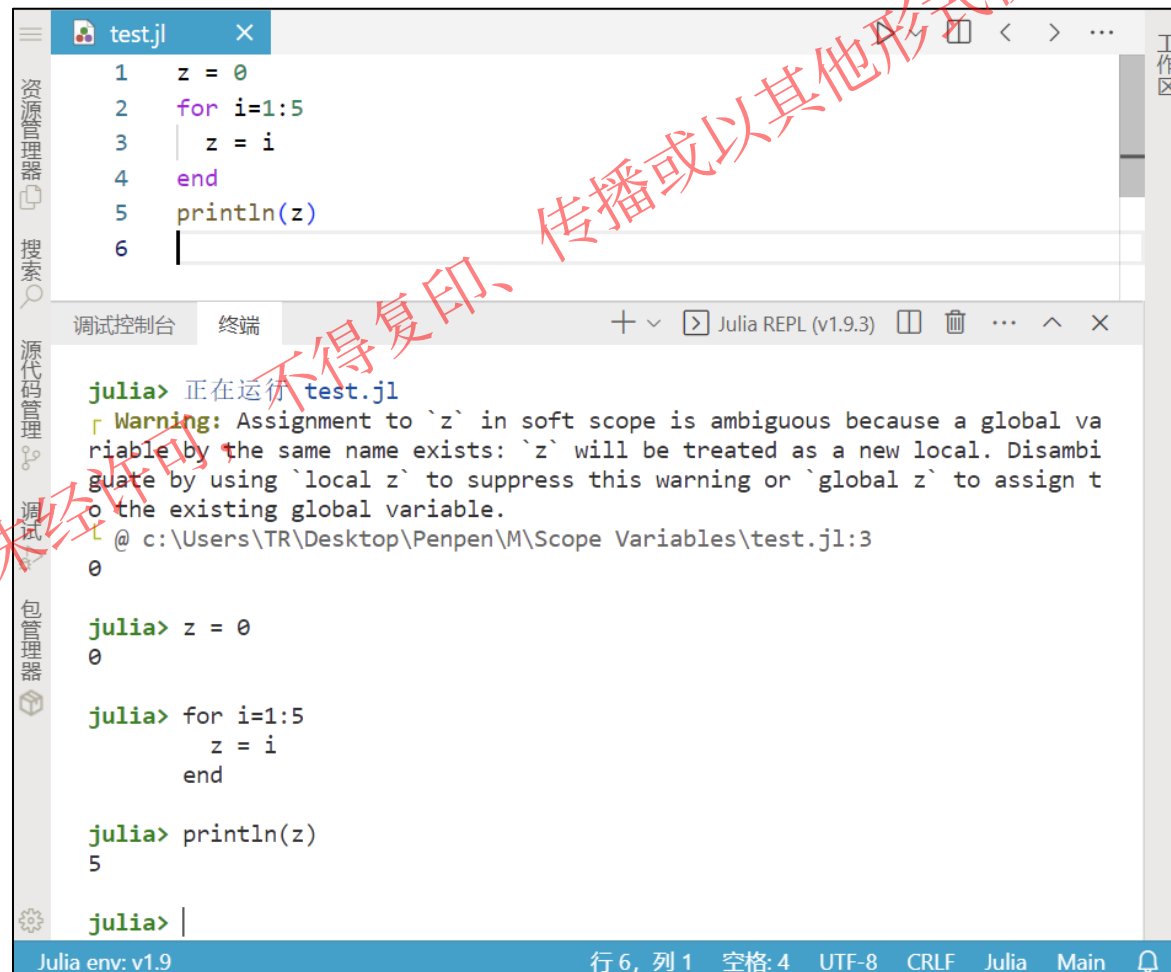
语言	作用域类型
Matlab	全局，局部作用域
Python	L(local), G(global), E(enclosing), B(built-in)
C	代码块作用域，文件作用域，函数原型作用域，函数作用域
C++	全局，函数，块作用域，类作用域...

1.作用域概念和意义

解决 Julia 使用过程中由于
作用域导致的问题

帮助用户从 Matlab 顺利的切换到 Syslab

深入理解 Julia 的作用域, 了解问题的原因



The screenshot shows a Julia REPL window with a script named `test.jl` open. The script contains the following code:

```
1 z = 0
2 for i=1:5
3     z = i
4 end
5 println(z)
6
```

The REPL output shows the following:

```
julia> 正在运行 test.jl
Warning: Assignment to `z` in soft scope is ambiguous because a global variable by the same name exists: `z` will be treated as a new local. Disambiguate by using `local z` to suppress this warning or `global z` to assign to the existing global variable.
@ c:\Users\TR\Desktop\Penpen\M\Scope Variables\test.jl:3
0

julia> z = 0
0

julia> for i=1:5
        z = i
    end

julia> println(z)
5

julia>
```

The warning message indicates that the assignment to `z` in the soft scope is ambiguous because a global variable with the same name exists. The warning suggests using `local z` to suppress the warning or `global z` to assign to the existing global variable.

- 为什么会在脚本中运行会产生该 warning 信息?
- 为什么在脚本和命令行中运行结果不同?
- 如何避免这样的问题?

02 Julia 的变量作用域

未经过许可，不得复印、传播或以其他形式使用

@苏州同元软控信息技术有限公司版权所有，侵权必究

2. Julia的变量作用域

在 Julia 中主要有两种作用域，全局作用域 与 局部作用域，可用关键字 `global`，`local` 声明。

结构	作用域类型	允许使用在
<code>module</code> , <code>baremodule</code> , <code>REPL</code>	全局	全局
<code>struct</code>	局部（软）	全局
<code>for</code> , <code>while</code> , <code>try</code>	局部（软）	全局, 局部
<code>macro</code>	局部（硬）	全局
函数, <code>do</code> 语句块, <code>let</code> 语句块	局部（硬）	全局, 局部

注意：同一个作用域内，每个变量的作用必须是唯一的，不能说在第一个语句是读取父作用域的同名变量值，第二个语句就因为赋值变成了该作用域的局部变量。

2. Julia的变量作用域 – 全局作用域

- 每一个模块有自己的全局作用域，但是没有一个在所有模块中都起作用的统一的全局作用域。模块内在所有函数定义外部赋值的变量为全局变量。模块内的任何位置用 `global` 关键字声明的变量为该模块的全局变量。
- 在命令行环境中定义的变量属于命令行环境的全局作用域，实际是 Main 模块的全局作用域。
- 为了访问其它模块的全局变量，可以用 `using` 或者 `import` 引入，也可以用“模块名.变量名”格式。

示例：每个模块有自己全局作用域，访问可以采用import引用

```
module Amod
    x = 1
    foo() = x
end
import .Amod
x = -1
println(Amod.foo())
## 1
```

最后一个语句调用 `Amod.foo()` 时用到的是模块 `Amod` 的全局变量 `x`，这是定义 `foo()` 时包含函数定义的代码中的 `x` 变量。
赋值为-1的变量 `x` 是命令行的全局变量，这是调用 `foo()` 时所处环境中的变量。

2. Julia的变量作用域 – 局部作用域

许多代码块结构都会产生一个局部作用域，如函数定义、for 循环等。局部作用域都可以读写其定义环境所处作用域（称为父作用域）的变量，但有例外：

- 如果在局部作用域中用 local 声明了变量，则对该变量的修改不会影响父作用域的变量。
- 局部作用域不是命名空间，所以内层可以访问外层变量，但是外层无论如何不能访问内层作用域的局部变量。
- 如果希望在局部作用域修改**全局**变量：
 - 非交互式环境：需要在局部作用域中用 global 声明该变量；
 - 可交互式环境：硬作用域需要用 global 声明该变量，软作用域不需要。

根据是否允许使用相同的变量名覆盖全局变量，局部作用域分成硬局部作用域和软局部作用域。

2. Julia的变量作用域 – 局部作用域

局部作用域：
局部变量无法修改全局变量

问题：在交互式环境中赋值操作不方便

在交互式环境中：
是否允许使用相同的
变量名覆盖全局变量



硬作用域：硬作用域中的变量无法覆盖全局变量

软作用域：在交互式环境中允许使用相同的变量名覆盖全局变量

The image shows two side-by-side screenshots of the Julia REPL (Read-Eval-Print Loop) interface. The left window shows Julia version 1.4.2 (2020-05-23). The right window shows Julia version 1.9.3 (2023-08-24). Both windows display the same code snippet: `z = 0; for i=1:5; z = i; end; println(z)`. In the left window (v1.4.2), the output is `0`. In the right window (v1.9.3), the output is `5`. This demonstrates that in the newer version, the local variable `z` successfully overwrote the global variable `z` within the scope of the loop, which is a change in the language's scoping rules.

2. Julia的变量作用域 – 局部作用域：硬作用域

硬作用域里的变量不会对全局作用域里的变量产生影响

● function

```
function foo()  
    x=2  
    return x  
end  
x=1  
foo()  
@show x
```

```
调试控制台 终端 Julia REPL (v1.9.3)  
  
julia> function foo()  
           x=2  
           return x  
       end  
foo (generic function with 1 method)  
  
julia> x=1  
1  
  
julia> foo()  
2  
  
julia> @show x  
x = 1  
1
```

● let结构

```
z = -1  
let x = 1, y = 2  
    z = x + y  
    @show z  
end  
@show z
```

```
调试控制台 终端 Julia REPL (v1.9.3)  
  
julia> z = -1  
-1  
  
julia> let x = 1, y = 2  
           z = x + y  
           @show z  
       end  
z = 3  
3  
  
julia> @show z  
z = -1  
-1
```

2. Julia的变量作用域 – 局部作用域：软作用域

软作用域里的变量可能对全局作用域里的变量产生影响：

- 如果全局变量 z 是**未定义**，最终此次赋值会在该作用域创建一个名为 z 的新局部变量，见示例1。
- 如果全局变量 z 是**已定义**：
 - 在**非交互**的上下文（文件、eval、include）中，会打印一个有歧义警告，同时创建一个新局部变量，见示例2；
 - 在**交互**的上下文（REPL, notebooks）中，会向全局变量 z 赋值，见示例3。

示例1

```
julia> @isdefined(z)
false

julia> for i = 1:5
    z = i
end

julia> @isdefined(z)
false
```

示例2

```
z = 0
for i = 1:5
    z = i
end
println(z)
```

```
调试控制台 终端 + Julia REPL (v1.9.3)
julia> 正在运行 test.jl
Warning: Assignment to `z` in soft scope is ambiguous because a global variable by the same name exists: `z` will be treated as a new local. Disambiguate by using `local z` to suppress this warning or `global z` to assign to the existing global variable.
@ c:\Users\TR\Desktop\Penpen\M\PreloadMPackages\TyPlot\2-D and 3-D Plots\Discrete Data Plots\stem\test.jl:3
0
```

示例3

```
julia> z = 0
0

julia> for i = 1:5
    z = i
end

julia> println(z)
5
```

2. Julia的变量作用域 – 局部作用域嵌套

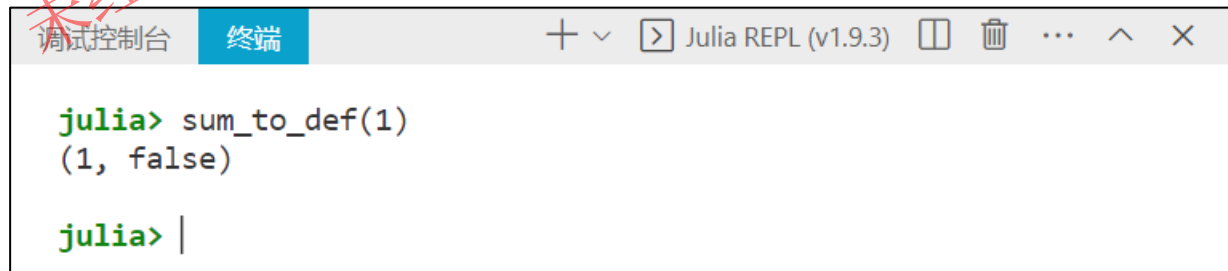
- 内层作用域寻找外层作用域的变量可以找到。
- 外层作用域寻找内层作用域的变量不可以找到。

示例：

for 循环语句中可以找到 function 中定义的变量 s

function 中无法找到 for 循环中定义的变量 t

```
function sum_to_def(n)
    s = 0 # new local
    for i = 1:n
        t = s + i # new local `t`
        s = t # assign existing local `s`
    end
    return s, @isdefined(t)
end
sum_to_def(1)
```



The screenshot shows a terminal window titled 'Julia REPL (v1.9.3)'. It contains the following text:

```
julia> sum_to_def(1)
(1, false)

julia> |
```

03

避免作用域引起的问题

@苏州同元软控信息技术有限公司版权所有，侵权必究

未经许可，不得复印、传播或以其他方式使用

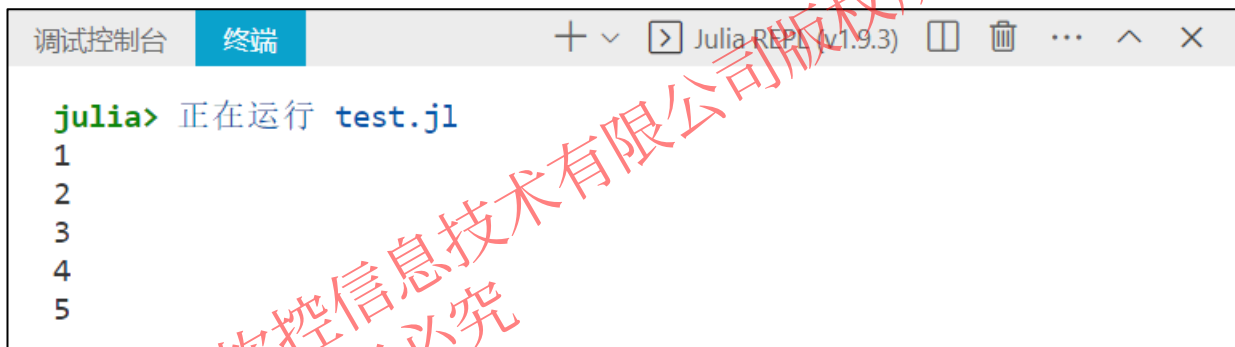
2.避免作用域引起的问题

1. 如何在局部作用域使用全局变量？

新建变量：

在局部作用域创建局部变量，将全局变量赋值给局部变量。

```
s = 0 # global `s`  
for i = 1:5  
    y = s + i # new local `y`  
    println(y)  
end
```



调试控制台 终端 + v > Julia REPL (v1.9.3) □ □ ... ^ x

```
julia> 正在运行 test.jl  
1  
2  
3  
4  
5
```

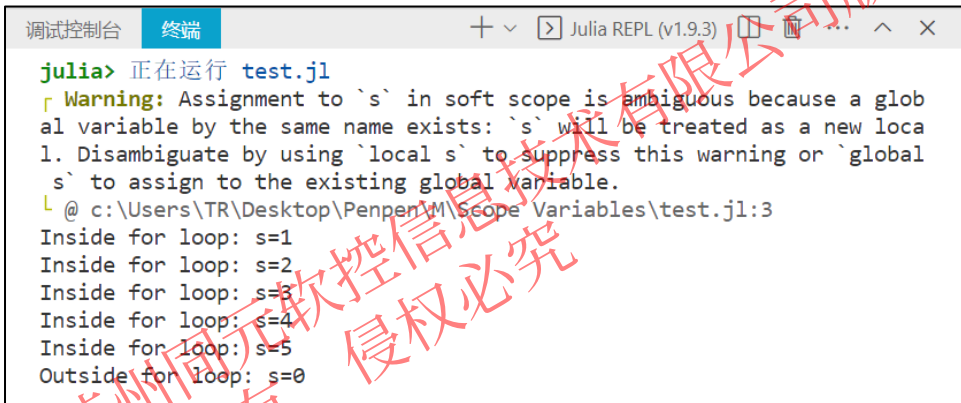

2.避免作用域引起的问题

2. 如何在局部作用域修改全局变量？

使用 local 和 global 标识符：

能够帮助 Julia 准确判断用户想使用的变量所在的作用域、提高代码的可读性。

```
s = 0 # global `s`  
for i = 1:5  
    s = i # new local `s`  
    println("Inside for loop: s=$s")  
end  
println("Outside for loop: s=$s")
```



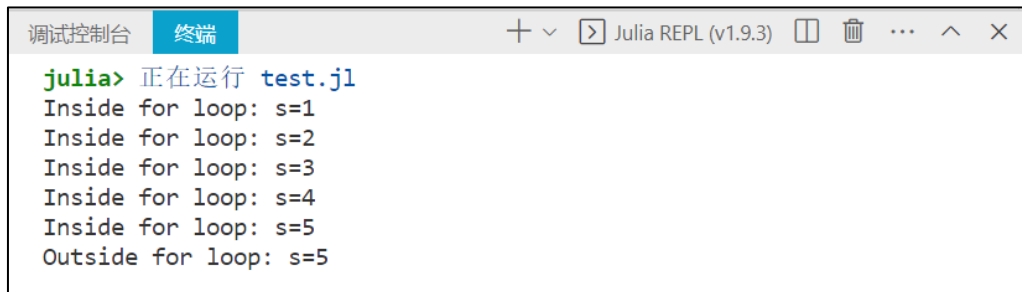
调试控制台 终端

Julia REPL (v1.9.3)

```
julia> 正在运行 test.jl  
[ Warning: Assignment to `s` in soft scope is ambiguous because a glob  
al variable by the same name exists: `s` will be treated as a new loca  
l. Disambiguate by using `local s` to suppress this warning or `global  
s` to assign to the existing global variable.  
L @ c:\Users\TR\Desktop\Penpen\M\Scope Variables\test.jl:3  
Inside for loop: s=1  
Inside for loop: s=2  
Inside for loop: s=3  
Inside for loop: s=4  
Inside for loop: s=5  
Outside for loop: s=0
```



```
s = 0 # global `s`  
for i = 1:5  
    global s  
    s = i # assign global `s`  
    println("Inside for loop: s=$s")  
end  
println("Outside for loop: s=$s")
```



调试控制台 终端

Julia REPL (v1.9.3)

```
julia> 正在运行 test.jl  
Inside for loop: s=1  
Inside for loop: s=2  
Inside for loop: s=3  
Inside for loop: s=4  
Inside for loop: s=5  
Outside for loop: s=5
```

2.避免作用域引起的问题

3. 如何避免全局作用域的和软局部作用域里的变量相互影响?

封装成函数：

脚本文件封成函数能够隔绝全局作用域，避免全局作用域的和软局部作用域里的变量相互影响。

```
s = 0 # global `s`
for i = 1:5
    s = s + i # local `s`
    println("Inside for loop: s=$s")
end
println("Outside for loop: s=$s")
```



```
function foo()  
  s = 0 # local `s`  
  for i = 1:5  
    s = s + i # assign existing local `s`  
    println("Inside for loop: s=$s")  
  end  
  println("Outside for loop: s=$s")  
end  
foo()
```

```

julia> 正在运行 test.jl
└─ Warning: Assignment to `s` in soft scope is ambiguous because a global variable by the same name exists: `s` will be treated as a new local variable. Disambiguate by using `local s` to suppress this warning or `global s` to assign to the existing global variable.
└─ @ c:\Users\TR\Desktop\Penpen\M\Scope Variables\test.jl:3
ERROR: UndefVarError: `s` not defined
Stacktrace:
 [1] top-level scope
      @ c:\Users\TR\Desktop\Penpen\M\Scope Variables\test.jl:3

```

```
调试控制台 终端 + > Julia REPL (v1.9.3) [ ] [X] ... ^ X
julia> 正在运行 test.jl
Inside for loop: s=1
Inside for loop: s=3
Inside for loop: s=6
Inside for loop: s=10
Inside for loop: s=15
Outside for loop: s=15
```

建立知识规范，营造协同生态
积累工业模型，发展可控平台
融入中国创新，打造先进软件

感谢聆听